DUALE HOCHSCHULE SACHSEN

Verteilte Systeme

Wintersemester 2026

Dr. Michael Hofmann

DHSN: Glauchau

10.09.2026

Allgemeine Informationen

Ablauf

- 6 Termine mit je 2x2 LVS (2x90 Minuten) = 24 LVS

$\rightsquigarrow$ jeweils „Vorlesung“ + Beispiele und ggf. Übungsaufgaben

- Vorlesungsmaterialen $\rightsquigarrow$ OPAL: Theorie der Betriebssysteme und verteilten Systeme
- Abschlussklausur: 120 Minuten $\rightsquigarrow$ $\frac{1}{3}$ für Verteilte Systeme

Literatur

- Andrew S. Tanenbaum und Maarten van Steen. *Verteilte Systeme: Grundlagen und Paradigmen.* 2. Ausgabe, Addison Wesley, 2007.

Themenüberblick (aktueller Plan)

(1) 31.08.2026
- Einführung

(2) 10.09.2026
- Architekturen

(3) 14.09.2026
- Prozesse

(4) 14.10.2026
- Kommunikation

(5) 27.10.2026
- Synchronisation

(6) 02.11.2026
- Konsistenz und Replikation
- Fehlertoleranz
- Wiederholung, Fragen & Antworten

Inhalt

1. Einführung
2. Architekturen
3. Prozesse
4. Kommunikation
5. Synchronisation
6. Konsistenz und Replikation
7. Fehlertoleranz

Inhalt

1. Einführung
2. Architekturen
3. Prozesse
4. Kommunikation
5. Synchronisation
6. Konsistenz und Replikation
7. Fehlertoleranz

Inhalt

1. Einführung
2. Architekturen
3. Prozesse
4. Kommunikation
5. Synchronisation
6. Konsistenz und Replikation
7. Fehlertoleranz

Architekturen für verteilte Systeme

- Ziel von (IT-)Architekturen ist die Reduktion von Komplexität
- Architekturen gegeben Organisation des verteilten Systems vor

⇝ Verschiedene Sichten auf Organisation möglich, z. B. logisch und physisch

Softwarearchitektur (logische Sicht)

- *Aus welchen Softwarekomponenten besteht das verteilte System?*

⇝ Aufteilungs- und Verortungsentscheidungen für konkrete Funktionalitäten

- Prinzipien zur Entscheidungsfindung ⇝ *SOLID*, *KISS*, *DRY*, *YAGNI*, ...

Systemarchitektur (physische Sicht)

- *Wie werden Softwarekomponenten auf reale Computer verteilt?*
- Praxis erlaubt zumeist eine Vielzahl an möglichen Verteilungen
- Grundlegende Organisationsformen ⇝ zentralisiert, dezentralisiert, hybrid

Inhalt

2. Architekturen
Architekturstile
Systemarchitekturen
Middlewares

Architekturstile

Wieso ist Softwarearchitektur notwendig und welche Bestandteile umfasst sie?

Architekturstile

Wieso ist Softwarearchitektur notwendig und welche Bestandteile umfasst sie?

⇝ Entscheidend für die erfolgreiche Entwicklung großer Systeme

Bestandteile

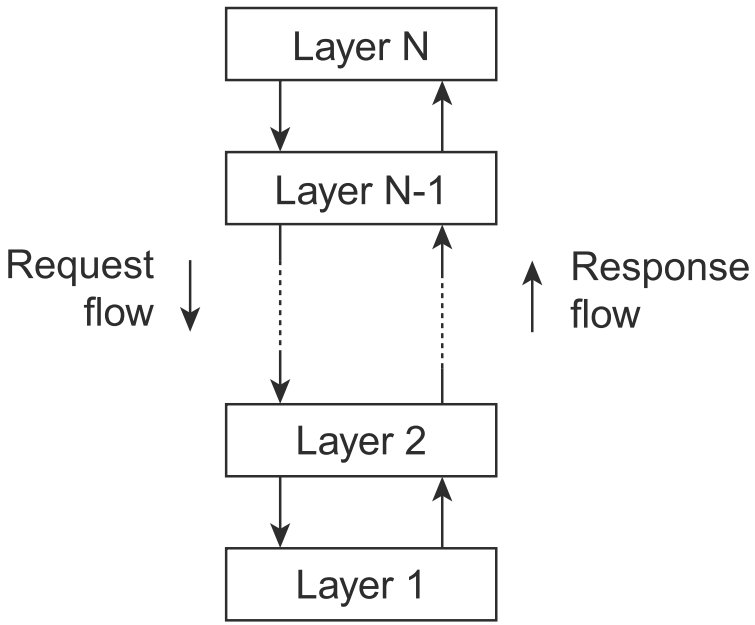
- Komponenten ⇝ modulare Einheit mit definierten Schnittstellen
- Verbindungen zw. Komponenten ⇝ Kommunikation und Koordination
- Datenaustausch zw. Komponenten ⇝ Kooperation
- Konfiguration zu einem System

Architekturstile für verteilte Systeme

1. Schichtenarchitekturen
2. Objektbasierte Architekturen
3. Ereignisbasierte Architekturen
4. Datenzentrierte Architekturen

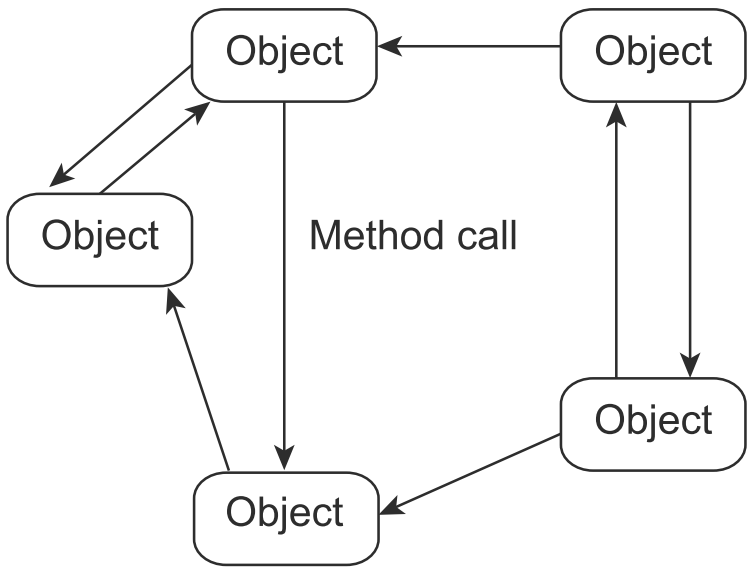
Schichtenarchitekturen

- Komponenten sind in Schichten organisiert
- Komponente in Schicht L_i kann nur Komponenten in Schicht L_{i-1} aufrufen
- Hohe Verbreitung im Bereich Netzwerke
- Kontrollfluss von Schicht zu Schicht
  $\rightsquigarrow$ Anfragen nach *unten*, Antworten nach *oben*



Objektbasierte Architekturen

- Jede Komponente entspricht einem *Objekt*
- Komponenten sind mittels (verteiltem) Prozeduraufruf verbunden
- Architekturstil passt mit dem Client-Server-Modell zusammen



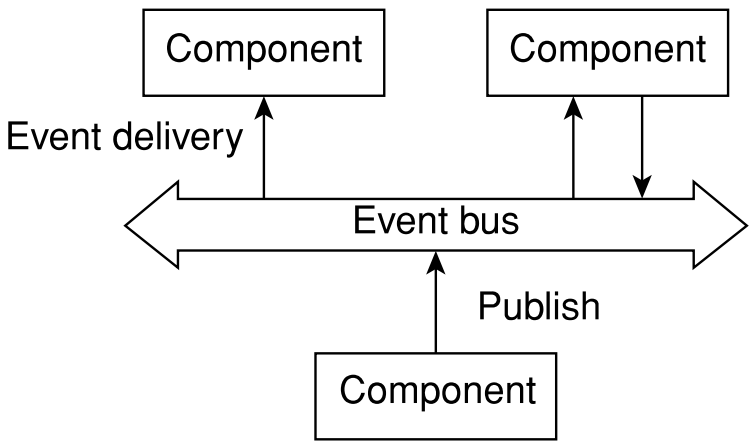
Ereignisbasierte Architekturen

- Komponenten kommunizieren durch Ereignisse (ggf. inkl. Daten)
- Realisierung in Form von sog. Publish/Subscribe-Systemen

⇝ Komponenten veröffentlichen und abonnieren Ereignisse

⇝ Middleware übernimmt Zuordnung und Weitergabe der Ereignisse

- Hauptvorteil: Lockere Koppelung der Komponenten



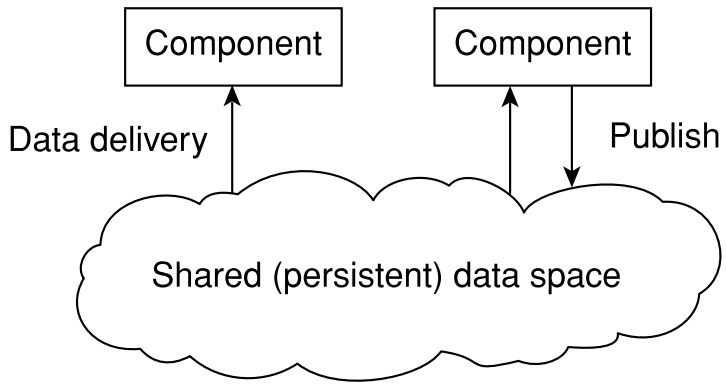
Datenzentrierte Architekturen

- Komponenten kommunizieren durch ein gemeinsamen Datenspeicher
- Datenspeicher können passiv oder aktiv sein
- Beispiel: Netzwerkanwendungen basierend auf verteilten Dateisystemen

Shared data spaces

- Kombination aus ereignisbasierter und datenzentrierter Architektur

\+ Zeitliche Entkoppelung



Inhalt

2. Architekturen
- Architekturstile
- Systemarchitekturen
- Middlewares

Systemarchitekturen

Welche Entscheidungen müssen (nach der Erstellung einer Softwarearchitektur) noch für die konkrete Realisierung eines verteilten Systems getroffen werden?

Systemarchitekturen

Welche Entscheidungen müssen (nach der Erstellung einer Softwarearchitektur) noch für die konkrete Realisierung eines verteilten Systems getroffen werden?

- Welche Softwarekomponenten werden konkret benötigt?
- Wie sollen Softwarekomponenten miteinander interagieren?
- Wo werden Softwarekomponenten (physisch) platziert?

Grundlegende Organisationsformen

1. Zentralisierte Architekturen
2. Dezentralisierte Architekturen
3. Hybride Architekturen

Zentralisierte Architekturen

Welches Rollenmodell liegt vielen verteilten Systemen zugrunde?

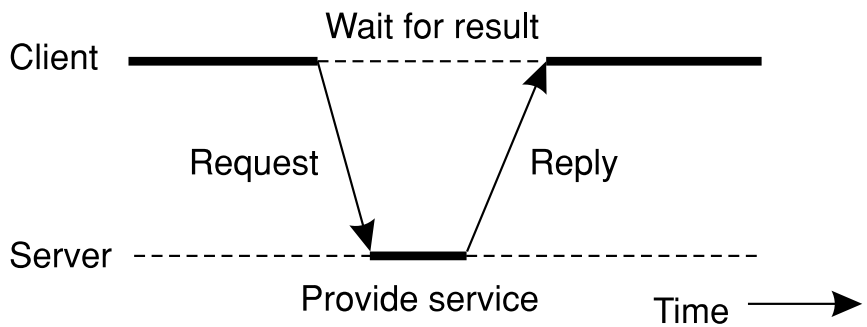
Zentralisierte Architekturen

Welches Rollenmodell liegt vielen verteilten Systemen zugrunde?

Client-Server-Modell

- ⇝ Rollenmodell zur Beherrschung der Komplexität von verteilten Systemen
- Aufteilung der Komponenten/Prozesse in zwei Gruppen:
  1. Server bieten bestimmte Dienste an, z. B. Dateisystem, Datenbank
  2. Clients fordern Dienste durch Anfrage an und wartet auf Antwort

Anfrage-Antwort-Verhalten



Zentralisierte Architekturen

Welches Rollenmodell liegt vielen verteilten Systemen zugrunde?

Client-Server-Modell

- ⇝ Rollenmodell zur Beherrschung der Komplexität von verteilten Systemen
- ▶ Aufteilung der Komponenten/Prozesse in zwei Gruppen:
  1. Server bieten bestimmte Dienste an, z. B. Dateisystem, Datenbank
  2. Clients fordern Dienste durch Anfrage an und wartet auf Antwort

Verbindungslose Protokolle

- ▶ Für zuverlässige Netzwerke (LANs)
- \+ Geringer Overhead, schnelle Kommunikation
- − Zusatzaufwand bei Verlust oder Beschädigung von Nachrichten

Verbindungsorientierte Protokolle

- ▶ Für unsichere Netzwerke (WANs)
- ▶ Internetprotokolle nutzen TCP
- − Overhead durch Verbindungsaufbau
- \+ Performanceverbesserung durch Mehrfachnutzung und Kompression

Anwendungsschichten

Lässt sich immer eine eindeutig Aufteilung in Client und Server finden?

Anwendungsschichten

Lässt sich immer eine eindeutig Aufteilung in Client und Server finden?

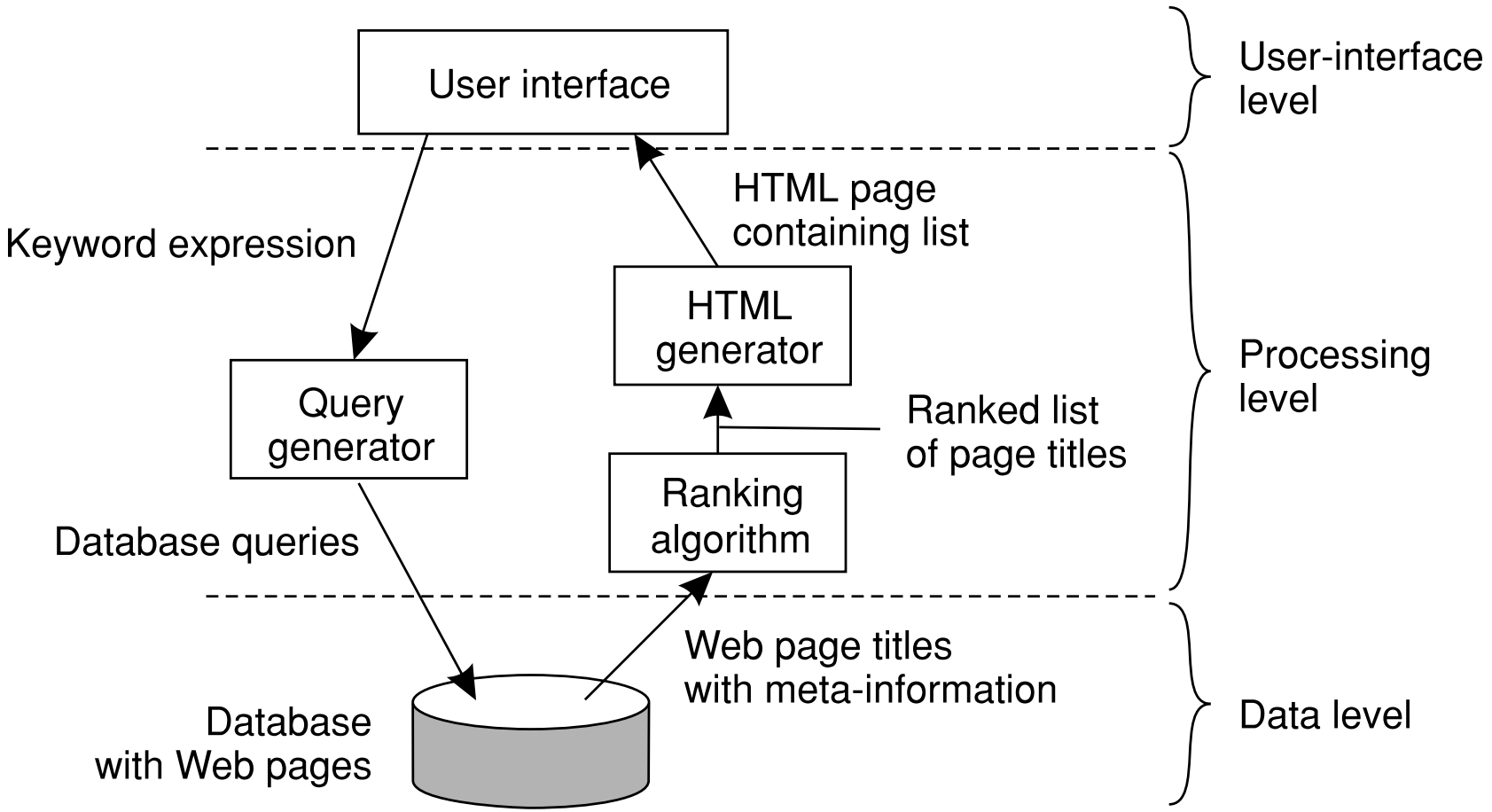
- Insbesondere in komplexen Systemen können sich die Rollen überlagern

⇝ Komponenten können sowohl Server als auch Client sein

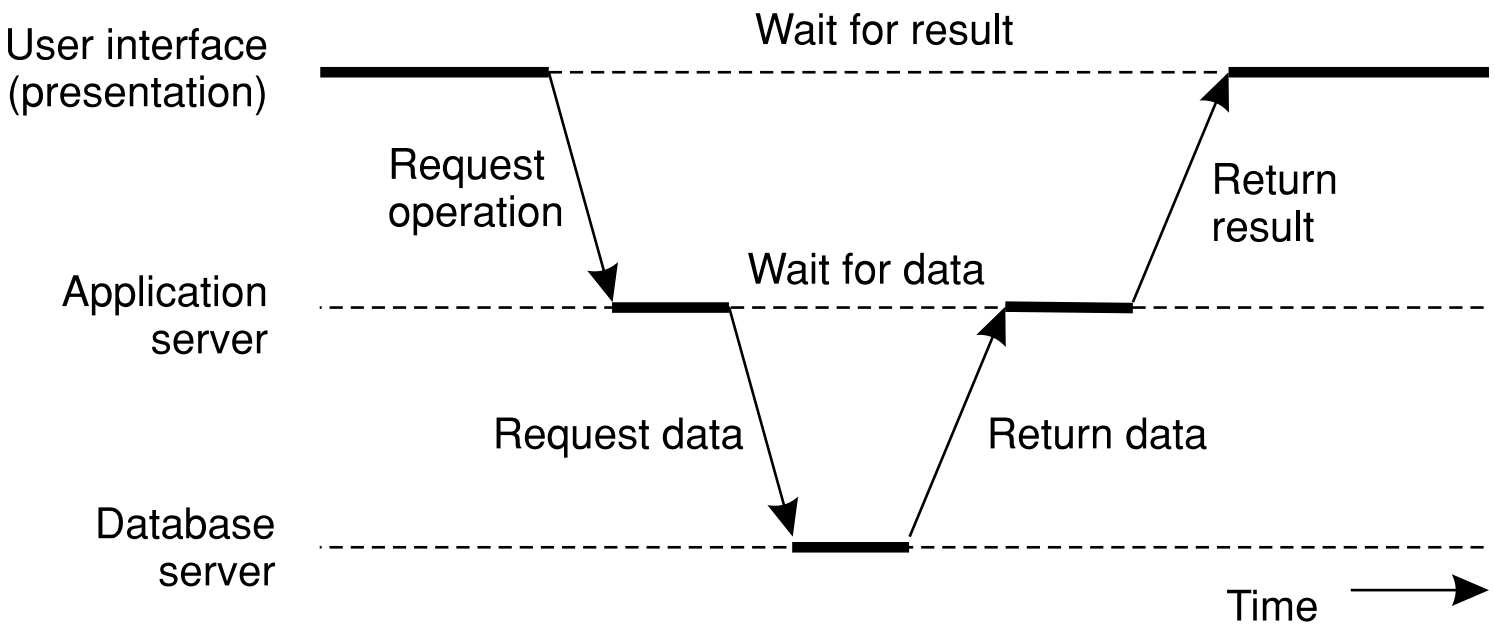
Drei-Schichten-Architektur für (interaktive) Anwendungen

1. Nutzerschnittstelle ⇝ meist Client
   - Realisiert Dateneingabe und -ausgabe des Nutzers, z. B. textuell, grafisch
   - Kann bereits lokale Verarbeitungen enthalten
2. Verarbeitung ⇝ meist Server & Client
   - Enthält die Kernfunktionalität der Anwendung
   - Wenig standardisierte Eigenschaften ⇝ abhängig von der jeweiligen Anwendung
3. Datenhaltung ⇝ meist Server
   - Verwaltung der eigentlichen Daten, z. B. Datenbank, Dateisystem
   - Unterstützt Verarbeitungsschicht mit persistenten Informationen

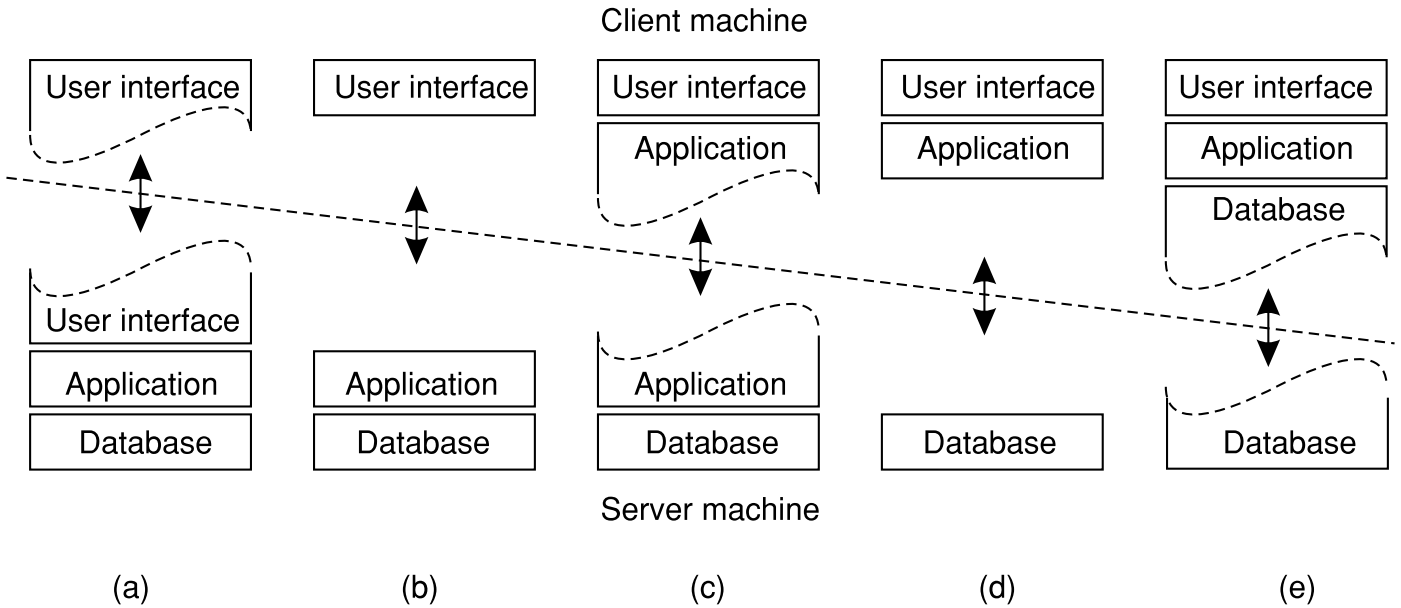
Beispiel: Internetsuchmaschine



Server agieren als Client



Fine Clients vs. Fat Clients



(a) Client enthält Teile der Nutzerschnittstelle & Anwendung kann diese steuern

(b) Client enthält komplettes grafisches Frontend, aber führt keine Verarbeitung aus

(c) Client führt z. B. Überprüfung von Nutzereingaben aus

(d) Client führt komplette grafische Anwendung aus und kommuniziert mit Datenbank

(e) Client speichert zusätzlich zu (d) noch Daten lokal

Dezentralisierte Architekturen

Horizontale vs. vertikale Verteilung

- Bisherige zentralisierte Drei-Schicht-Architektur
  - Vertikale Aufteilung in logische Komponenten inkl. Verteilung auf Computer
  + Spezialisierung der Computer für jeweilige Funktionalitäten
- Horizontale Verteilung führt zu dezentralisierter Architektur
  - Fokus auf Verteilung von Clients und Servern selbst
  - Aufspaltung in logisch äquivalente Teile
  - Jedes Teil arbeitet auf eigenem Datenausschnitt $\rightsquigarrow$ Lastverteilung

Peer-to-Peer Architekturen

- Aufteilung in gleichberechtigte Prozesse, sog. *Peers* (Client & Server gleichzeitig)

$\rightsquigarrow$ Symmetrische Interaktion zw. Prozessen

- Kommunikation über Overlay-Netzwerke $\rightsquigarrow$ Nachrichten werden weitergeleitet
  - Strukturierte Netze, z. B. mit Routing-Tabellen
  - Unstrukturierte Netze, z. B. mit Flooding oder Suchen

Strukturierte Peer-to-Peer Architekturen

- Einsatz: Dynamische Datenspeicherung verteilt über Peers/Knoten
- Overlay-Netzwerk wird durch deterministisches Verfahren aufgebaut
- Häufigstes Verfahren $\rightsquigarrow$ Verteilte Hashtabelle

Was sind normale Hashtabellen? Was sind verteilte Hashtabellen?

Strukturierte Peer-to-Peer Architekturen

- Einsatz: Dynamische Datenspeicherung verteilt über Peers/Knoten
- Overlay-Netzwerk wird durch deterministisches Verfahren aufgebaut
- Häufigstes Verfahren $\rightsquigarrow$ Verteilte Hashtabelle

Was sind normale Hashtabellen? Was sind verteilte Hashtabellen?

Verteilte Hashtabelle

- Datenobjekte und Knoten erhalten Schlüssel aus großem Bereich, z. B. 160-Bit
- Ziel: Effiziente & eindeutige Abbildung von Daten-Schlüssel $\rightarrow$ Knoten-Schlüssel

$\rightsquigarrow$ Anfrage an verteilte Hashtabelle entspricht Routing zum zuständigen Knoten

- Herausforderungen
  - Dynamisches Hinzufügen von Daten
  - Dynamisches Hinzufügen/Entfernen von Knoten
  - Datengrößen übersteigen Kapazitäten von einzelnen Knoten

Unstrukturierte Peer-to-Peer Architekturen

- Overlay-Netzwerk wird durch zufällige Algorithmen konstruiert
- Jeder Knoten führt eine (zufällig gebildete) Nachbarschaftsliste (Teilsicht)
- Datenobjekte werden zufällig auf Knoten verteilt
- Suche nach Daten $\rightsquigarrow$ meist durch Flooding, d. h. Nachbarn anfragen

Mitgliedschaftsmanagement

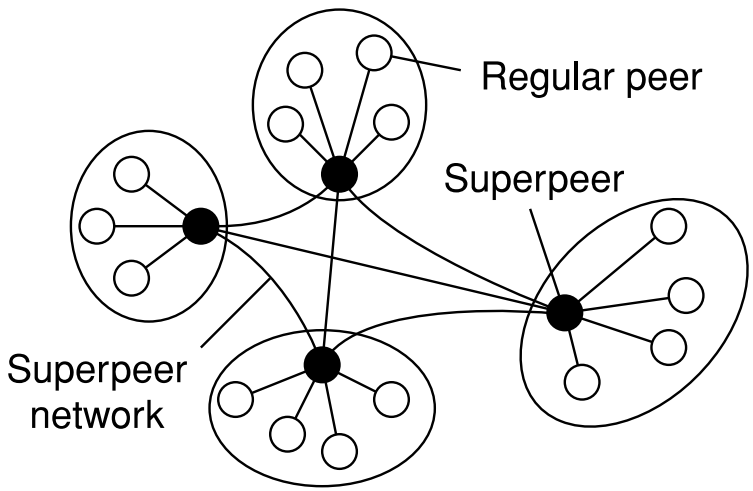
- Knoten verbinden sich über bekannte Einstiegspunkte
- Verlassen jederzeit ohne Abmeldung $\rightsquigarrow$ nicht mehr antwortende Nachbarn werden automatisch aus Nachbarschaftsliste entfernt

Herausforderungen der Nachbarschaftslisten

- Verhinderung von isolierten Gruppen
- Gefahr von Lastungleichgewicht wenn Knoten zu oft kontaktiert werden
- Methoden zum Verwerfen alter Einträge beeinflussen Netzwerkbalance

Superpeers

- Skalierbarkeit bei unstrukturierten Peer-to-Peer-Systemen beschränkt
- Aufgabe der Symmetrie durch Einführung von Spezialknoten $\rightsquigarrow$ Superpeers
- Superpeers speichern separate Indizes oder fungieren als Broker
- Neue Herausforderung: Wer darf Superpeer sein? (*Leader-Election-Problem*)



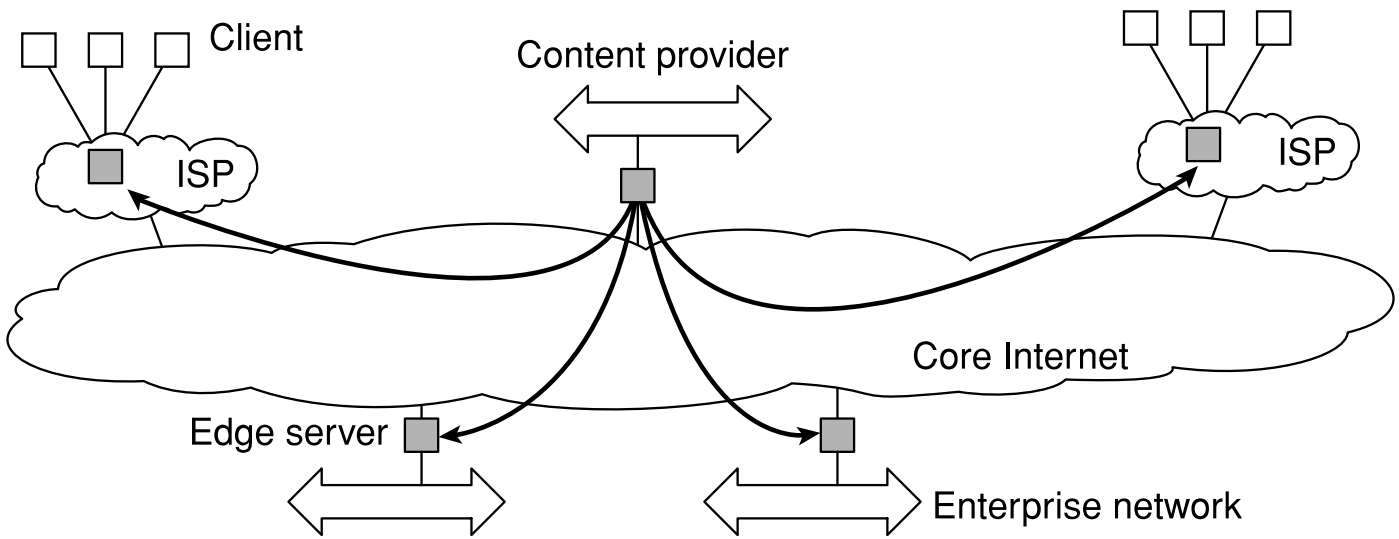
Hybride Architekturen

- Kombination verschiedener Architektureigenschaften in der Praxis

Edge-Server-Systeme

- Server werden am *Rand* von (Sub)Netzwerken positioniert
- Clients verbinden sich zu Edge-Servern ihres (Sub)Netzwerkes

Beispiel: Internet Service Provider (ISP)



Inhalt

2. Architekturen
- Architekturstile
- Systemarchitekturen
- Middlewares

Middlewares

Was ist eine Middleware und wozu gibt es sie?

Middlewares

Was ist eine Middleware und wozu gibt es sie?

- Schicht zwischen Anwendung und verteilter Platform
- Ziel: Verteilungstransparenz erreichen, z. B. Daten, Verarbeitung und Kontrolle werden vor der Anwendung verborgen

Architekturstile von Middlewares

- Objektbasiert, z. B. Common Object Request Broker Architecture (CORBA)
- Eventbasiert, z. B. Robot Operating System (ROS)

Vor- und Nachteile spezifischer Architekturstile

- Vorteil: Vereinfachung der Anwendungsentwicklung
- Nachteil: Einschränkungen, fehlende Flexibilität
- Beispiel: CORBA
  - zunächst nur Remote-Objektaufrufe
  - später Erweiterung um Nachrichtenübermittlung

Demo: Producer-Consumer mit ROS

```python
#!/usr/bin/env python

import rospy
from std_msgs.msg import String

def producer():
    pub = rospy.Publisher("chatter", String, queue_size=10)
    rospy.init_node("producer_node", anonymous=True)
    rate = rospy.Rate(1) # 1 Hz
    count = 0
    while not rospy.is_shutdown():
        message_str = f"Hello ROS {count}"
        rospy.loginfo(message_str)
        pub.publish(message_str)
        count += 1
        rate.sleep()

if __name__ == "__main__":
    try:
        producer()
    except rospy.ROSInterruptException:
        pass
```

```python
#!/usr/bin/env python

import rospy
from std_msgs.msg import String

def callback(data):
    msg = f"{rospy.get_caller_id()}: I heard {data.data}"
    rospy.loginfo(msg)

def consumer():
    rospy.init_node("consumer_node", anonymous=True)
    rospy.Subscriber("chatter", String, callback)
     # Keep Python from exiting until node is stopped
    rospy.spin()

if __name__ == "__main__":
    consumer()
```

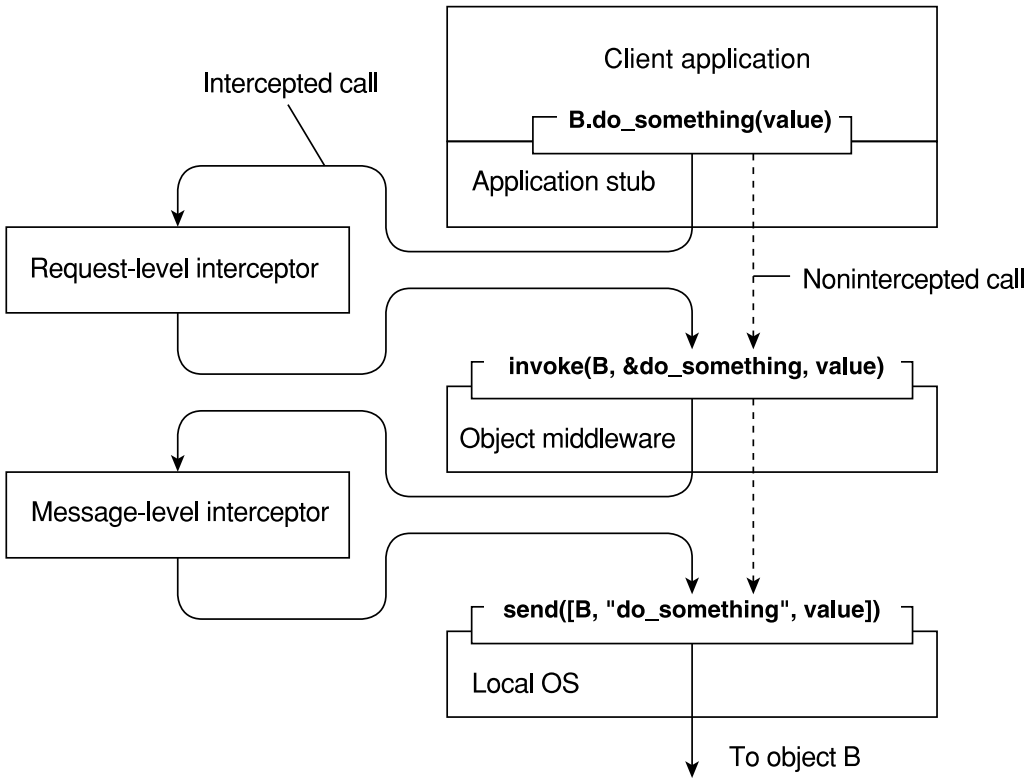
Anpassung von Middleware-Verhalten

Wie lässt sich Software (z. B. Middleware) gezielt erweitern oder anpassen?

Anpassung von Middleware-Verhalten

Wie lässt sich Software (z. B. Middleware) gezielt erweitern oder anpassen?

- Statisch ⇝ Daten: Parameter, Spezifikationen, ...
- Dynamisch ⇝ Code: Interceptoren, Callbacks, ...



Entwicklung adaptiver Software

- Umgebung von verteilten Anwendungen kann sich permanent ändern
- Adaption sollte von Middleware anstatt von Anwendung übernommen werden

⇝ Interceptoren sind ein Mechanismus zu Adaption von Middlewares

Techniken für adaptive Software

- Trennung von Verantwortlichkeiten (Separation of concerns)
  - Traditionelle Erstellung modularer Software

  ⇝ Herausforderung: Nicht-funktionale Anforderungen, z. B. Sicherheit, Performance und Zuverlässigkeit, lassen sich schwer klar trennen
- Computational Reflexion
  - Programme können sich selbst inspizieren und ihr Verhalten zur Laufzeit ändern

  ⇝ Nicht von jeder Programmiersprache unterstützt
- Komponentenbasierte Architektur
  - Gezielte Komposition von Einzelkomponenten (statisch oder dynamisch)

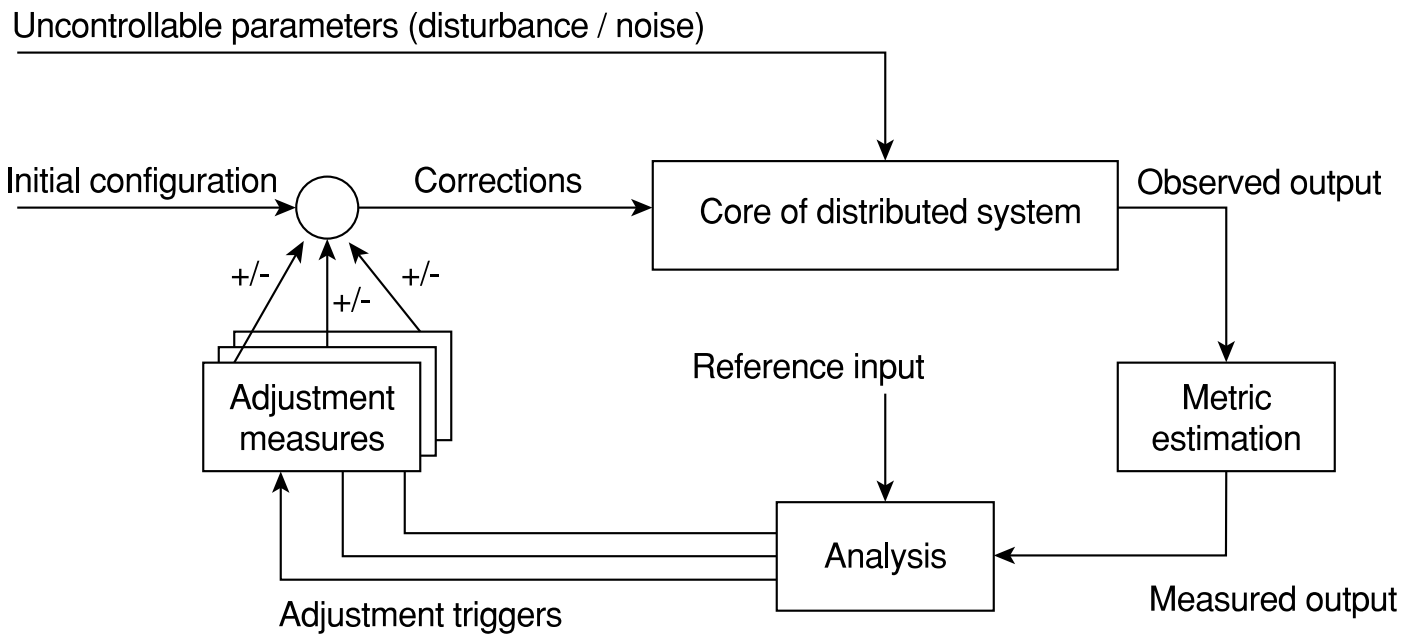
  ⇝ Erfordert sog. *Late Binding*, d. h. Nachladen von Modulen

Selbstmanagement in verteilten Systemen

- Verteilte Systeme müssen auf Umgebungsveränderungen reagieren können

⇝ Ausführungsverhalten und -verteilung muss sich selbstständig anpassen

Regelkreismodell / *Feedback Control Model*



Inhalt

1. Einführung
2. Architekturen
3. Prozesse
4. Kommunikation
5. Synchronisation
6. Konsistenz und Replikation
7. Fehlertoleranz

Inhalt

1. Einführung
2. Architekturen
3. Prozesse
4. Kommunikation
5. Synchronisation
6. Konsistenz und Replikation
7. Fehlertoleranz

Inhalt

1. Einführung
2. Architekturen
3. Prozesse
4. Kommunikation
5. Synchronisation
6. Konsistenz und Replikation
7. Fehlertoleranz

Inhalt

1. Einführung
2. Architekturen
3. Prozesse
4. Kommunikation
5. Synchronisation
6. Konsistenz und Replikation
7. Fehlertoleranz

Inhalt

1. Einführung
2. Architekturen
3. Prozesse
4. Kommunikation
5. Synchronisation
6. Konsistenz und Replikation
7. Fehlertoleranz